```cpp
#pragma once
#include <iostream>
#include <vector>
#include <cmath>
#include "matrix.hpp"

class capaAtencion {
private:
    size_t dimIn_;
    size_t dimK_;

    // Matrices de proyección entrenables
    Matriz Wq, Wk, Wv;

    // Momentos de Adam
    Matriz mWq, vWq;
    Matriz mWk, vWk;
    Matriz mWv, vWv;

    // Caché para Backward Pass
    Matriz X_cache;
    Matriz Q_cache, K_cache, V_cache;
    Matriz scores_cache;
    Matriz A_cache;

    double beta1 = 0.9;
    double beta2 = 0.999;
    double epsilon = 1e-8;

public:
    capaAtencion(size_t dimIn, size_t dimK)
        : dimIn_(dimIn), dimK_(dimK)
    {
        Wq = Matriz(dimK_, dimIn_);
        Wk = Matriz(dimK_, dimIn_);
        Wv = Matriz(dimK_, dimIn_);

        mWq = Matriz(dimK_, dimIn_, 0.0); vWq = Matriz(dimK_, dimIn_, 0.0);
        mWk = Matriz(dimK_, dimIn_, 0.0); vWk = Matriz(dimK_, dimIn_, 0.0);
        mWv = Matriz(dimK_, dimIn_, 0.0); vWv = Matriz(dimK_, dimIn_, 0.0);

        double limite = std::sqrt(6.0 / (dimIn_ + dimK_));
        Wq.aleatorizar(-limite, limite);
        Wk.aleatorizar(-limite, limite);
        Wv.aleatorizar(-limite, limite);
    }

   // Forward Pass con Máscara Causal
    Matriz forward(const Matriz& X) {
        X_cache = X;

        // 1. Proyecciones Q, K, V
        Q_cache = Wq.matmul(X); // (dimK x seqLen)
        K_cache = Wk.matmul(X); // (dimK x seqLen)
        V_cache = Wv.matmul(X); // (dimK x seqLen)

        // 2. Scores = (Q^T * K) / sqrt(dimK)
        Matriz scores = Q_cache.transpuesta().matmul(K_cache);
        double escala = std::sqrt(static_cast<double>(dimK_));

        for (size_t i = 0; i < scores.filas(); ++i) {
            for (size_t j = 0; j < scores.columnas(); ++j) {
                scores(i, j) /= escala;

                // --- MÁSCARA CAUSAL ---
                // Si la columna j está adelantada a la fila i (futuro), bloqueamos la
```

atención

```
                if (j > i) {
                    scores(i, j) = -1e9;
                }
            }
        }
        scores_cache = scores;

        // 3. Softmax por filas (Atención A)
        A_cache = Matriz(scores.filas(), scores.columnas());
        for (size_t i = 0; i < scores.filas(); ++i) {
            Matriz filaTemp(scores.columnas(), 1);
            for (size_t j = 0; j < scores.columnas(); ++j) {
                filaTemp(j, 0) = scores(i, j);
            }
            Matriz filaSoftmax = filaTemp.softmax();
            for (size_t j = 0; j < scores.columnas(); ++j) {
                A_cache(i, j) = filaSoftmax(j, 0);
            }
        }

        // 4. Salida contextualizada
        return V_cache.matmul(A_cache.transpuesta());
    }

    // Backward Pass con optimización Adam
    Matriz backward(const Matriz& gradSalida, double lr, int t) {
        Matriz dV = gradSalida.matmul(A_cache);
        Matriz dA = gradSalida.transpuesta().matmul(V_cache);

        Matriz dScores(scores_cache.filas(), scores_cache.columnas(), 0.0);
        double escala = std::sqrt(static_cast<double>(dimK_));

        for (size_t i = 0; i < A_cache.filas(); ++i) {
            for (size_t j = 0; j < A_cache.columnas(); ++j) {
                double suma = 0.0;
                for (size_t k = 0; k < A_cache.columnas(); ++k) {
                    double delta = (j == k) ? 1.0 : 0.0;
                    suma += dA(i, k) * A_cache(i, k) * (delta - A_cache(i, j));
                }
                dScores(i, j) = suma / escala;
            }
        }

        Matriz dQ = K_cache.matmul(dScores.transpuesta());
        Matriz dK = Q_cache.matmul(dScores);

        Matriz dWq = dQ.matmul(X_cache.transpuesta());
        Matriz dWk = dK.matmul(X_cache.transpuesta());
        Matriz dWv = dV.matmul(X_cache.transpuesta());

        actualizarParametro(Wq, dWq, mWq, vWq, lr, t);
        actualizarParametro(Wk, dWk, mWk, vWk, lr, t);
        actualizarParametro(Wv, dWv, mWv, vWv, lr, t);

        return Wq.transpuesta().matmul(dQ) +
               Wk.transpuesta().matmul(dK) +
               Wv.transpuesta().matmul(dV);
    }
    void guardarBinario(std::ofstream& out) const {
        Wq.guardarBinario(out);
        Wk.guardarBinario(out);
        Wv.guardarBinario(out);
    }
    void cargarBinario(std::ifstream& in) {
        Wq.cargarBinario(in);
```

```
        Wk.cargarBinario(in);
        Wv.cargarBinario(in);
    }

private:
    void actualizarParametro(Matriz& W, const Matriz& dW, Matriz& mW, Matriz& vW, double
lr, int t) {
        for (size_t i = 0; i < W.filas(); ++i) {
            for (size_t j = 0; j < W.columnas(); ++j) {
                double g = dW(i, j);
                mW(i, j) = beta1 * mW(i, j) + (1.0 - beta1) * g;
                vW(i, j) = beta2 * vW(i, j) + (1.0 - beta2) * (g * g);

                double m_hat = mW(i, j) / (1.0 - std::pow(beta1, t));
                double v_hat = vW(i, j) / (1.0 - std::pow(beta2, t));

                W(i, j) -= (lr * m_hat) / (std::sqrt(v_hat) + epsilon);
            }
        }
    }

};
```